



Trường ĐH CNTT – ĐHQG TP. HCM

NT521 - Lập trình an toàn & Khai thác lỗ hổng phần mềm

Phần 2 – Khai thác lỗ hổng phần mềm **Introduction to Exploitation**



- Khai thác phần mềm - Một số khái niệm cơ bản
- Khai thác phần mềm - Ôn tập 1 số kiến thức
 - Ngôn ngữ assembly – Hợp ngữ
 - Quản lý bộ nhớ
 - Big Endian vs Little Endian
 - C/C++ sang Assembly
 - Stack?
 - Instruction – Lệnh
- Lỗi hỏng tràn bộ đệm trên stack
 - Lỗi hỏng tràn bộ đệm: mục tiêu, ví dụ
 - Cách khắc phục



Yêu cầu:

- Assembly – Hợp ngữ, C/C++
- Hệ điều hành, quản lý tiến trình, bộ nhớ,...
- Các công cụ debug: GDB, GDB-PEDA, Immune Debugger, IDA, OllyDbg, Hopper Disassembler...
- Các ngôn ngữ lập trình: Python, Ruby, Go...
- Nguyên tắc lập trình an toàn
- Linux
- pwntools - CTF toolkit
- Ngăn chặn tấn công và Vượt qua ngăn chặn tấn công
- **Fuzzing:** (afl-fuzz, libFuzzer, honggfuzz), AddressSanitizer, Valgrind
- Đánh giá mã nguồn: đọc Source Code, Dịch ngược (Reverse Engineering)...





Wargame & Labs:

- <http://pwnable.kr/>
- <https://pwnable.tw/>
- <https://pwnable.vn/>
- <https://ctfs.me/>
- <https://www.root-me.org/>
- <http://ringzer0team.com/>
- <https://www.vulnhub.com/>
- <https://github.com/shellphish/how2heap>
- <https://ctf-wiki.org/pwn/linux/user-mode/environment/>



Các khái niệm cơ bản

Vulnerability – Lỗ hổng (danh từ): một lỗi trong bảo mật của hệ thống, có thể cho phép kẻ tấn công sử dụng hệ thống theo 1 cách khác với thiết kế của hệ thống.

Có thể bao gồm:

- Tác động đến tính sẵn sàng của hệ thống.
- Leo thang quyền.
- Điều khiển toàn bộ hệ thống trái phép.
- ...

Tên gọi khác: *security hole* hoặc *security bug*.

Khai thác lỗ hổng phần mềm

Các khái niệm cơ bản



Vulnerability – Common Vulnerabilities and Exposures (CVE)





Vulnerabilities (CVE)

OpenCVE > Vulnerabilities (CVE)

Select a tag

Filter by CVSS v3 score

Search in CVEs

Search

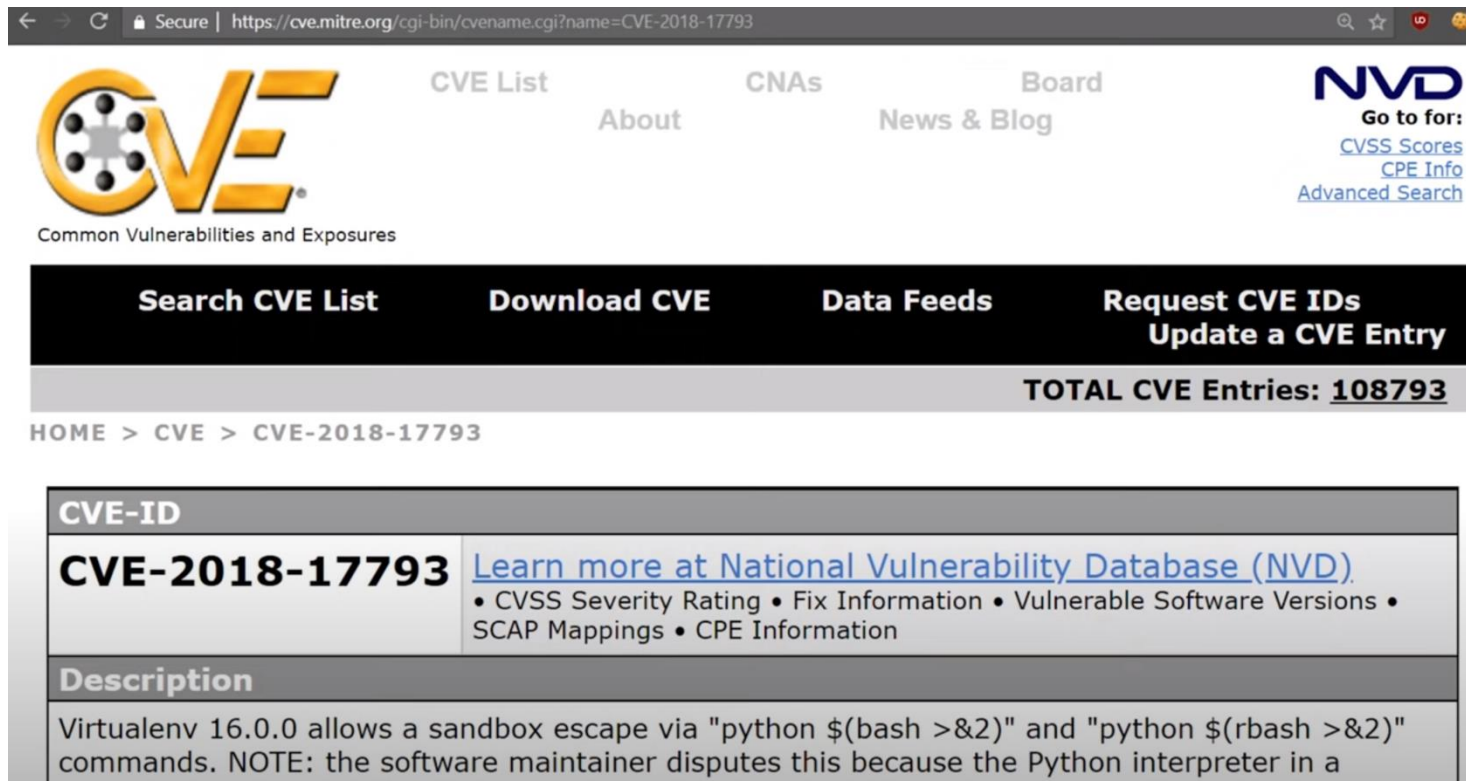
TOTAL

177301 CVE

CVE	Vendors	Products	Updated	CVSS v2	CVSS v3
CVE-2020-0618	1 Microsoft	1 Sql Server	2022-01-01	6.5 MEDIUM	8.8 HIGH
A remote code execution vulnerability exists in Microsoft SQL Server Reporting Services when it incorrectly handles page requests, aka 'Microsoft SQL Server Reporting Services Remote Code Execution Vulnerability'.					
CVE-2020-6380	2 Fedoraproject, Google	2 Fedora, Chrome	2022-01-01	6.8 MEDIUM	8.8 HIGH
Insufficient policy enforcement in extensions in Google Chrome prior to 79.0.3945.130 allowed a remote attacker who had compromised the renderer process to bypass site isolation via a crafted Chrome Extension.					
CVE-2020-6379	2 Fedoraproject, Google	2 Fedora, Chrome	2022-01-01	6.8 MEDIUM	8.8 HIGH
Use after free in V8 in Google Chrome prior to 79.0.3945.130 allowed a remote attacker to potentially exploit heap corruption via a crafted HTML page.					
CVE-2020-7217	1 Opensuse	1 Wicked	2022-01-01	5.0 MEDIUM	7.5 HIGH
An ni_dhcp4_fsm_process_dhcp4_packet memory leak in openSUSE wicked 0.6.55 and earlier allows network attackers to cause a denial of service by sending DHCP4 packets with a different client-id.					
CVE-2020-3934	1 Secom	2 Dr.id Access Control, Dr.id Attendance System	2022-01-01	7.5 HIGH	9.8 CRITICAL
TAIWAN SECOM CO., LTD., a Door Access Control and Personnel Attendance Management system, contains a vulnerability of Pre-auth SQL Injection, allowing attackers to inject a specific SQL command.					
CVE-2020-3933	1 Secom	2 Dr.id Access Control, Dr.id Attendance System	2022-01-01	5.0 MEDIUM	5.3 MEDIUM
TAIWAN SECOM CO., LTD., a Door Access Control and Personnel Attendance Management system, allows attackers to enumerate and exam user account in the system.					
CVE-2019-17061	1 Cypress	2 Psoc 4, Psoc 4 Ble	2022-01-01	6.1 MEDIUM	6.5 MEDIUM



Vulnerability – Common Vulnerabilities and Exposures (CVE)



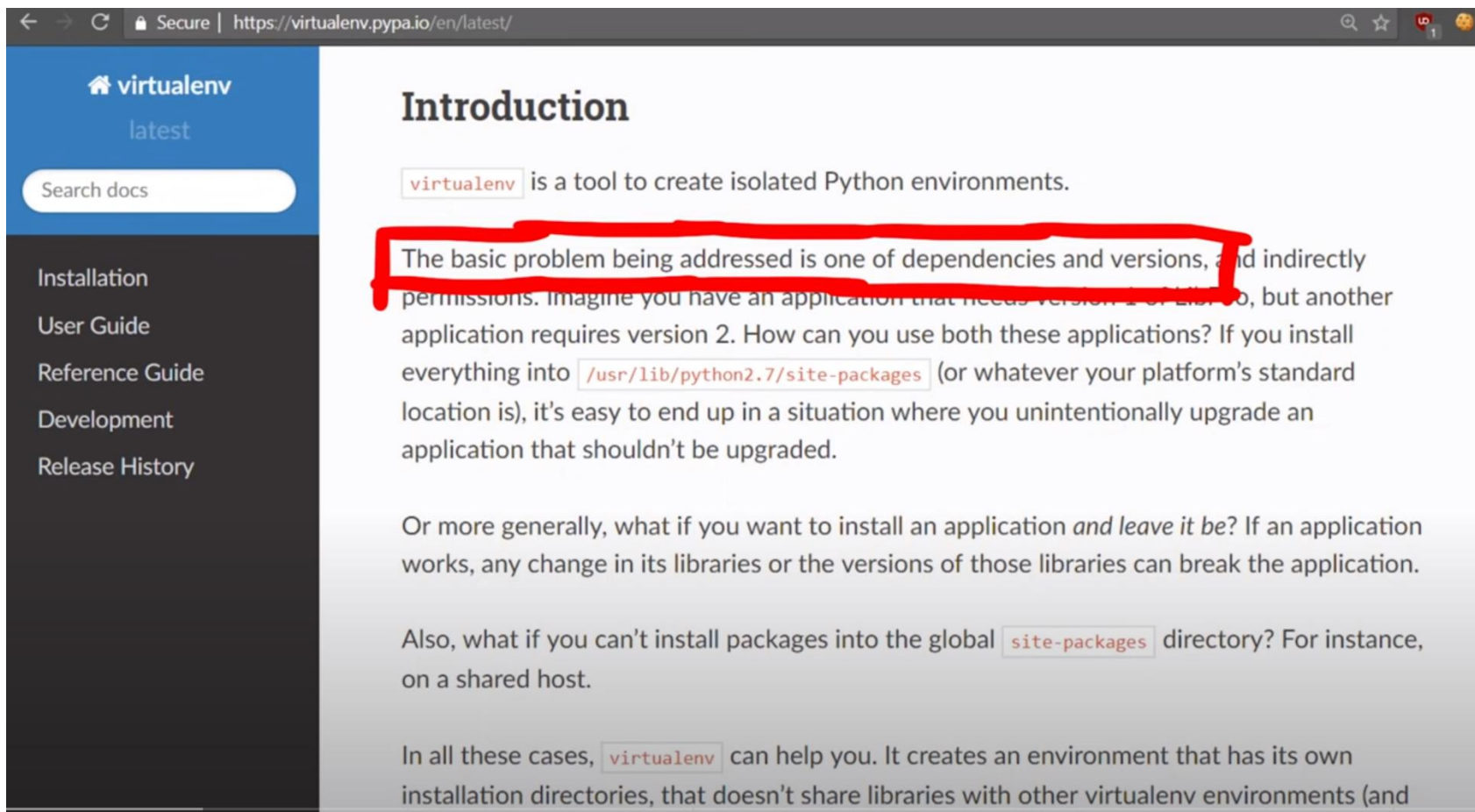
The screenshot shows the CVE Mitre website interface. At the top, there's a navigation bar with links like "CVE List", "About", "CNAs", "Board", and "News & Blog". The main header features the CVE logo and the text "Common Vulnerabilities and Exposures". To the right, there's a section for "NVD" (National Vulnerability Database) with links to "CVSS Scores", "CPE Info", and "Advanced Search". Below the header, there's a black bar with white text for "Search CVE List", "Download CVE", "Data Feeds", and "Request CVE IDs Update a CVE Entry". A grey bar below this shows "TOTAL CVE Entries: 108793". The breadcrumb trail reads "HOME > CVE > CVE-2018-17793". The main content area is a table with the following structure:

CVE-ID	
CVE-2018-17793	Learn more at National Vulnerability Database (NVD) • CVSS Severity Rating • Fix Information • Vulnerable Software Versions • SCAP Mappings • CPE Information
Description	
Virtualenv 16.0.0 allows a sandbox escape via "python \$(bash >&2)" and "python \$(rbash >&2)" commands. NOTE: the software maintainer disputes this because the Python interpreter in a	

Vulnerability – Common Vulnerabilities and Exposures (CVE)

CVE-ID	
CVE-2018-17793	Learn more at National Vulnerability Database (NVD) • CVSS Severity Rating • Fix Information • Vulnerable Software Versions • SCAP Mappings • CPE Information
Description	
Virtualenv 16.0.0 allows a sandbox escape via "python \$(bash >&2)" and "python \$(rbash >&2)" commands. NOTE: the software maintainer disputes this because the Python interpreter in a virtualenv is supposed to be able to execute arbitrary code.	
References	
Note: References are provided for the convenience of the reader to help distinguish between vulnerabilities. The list is not intended to be complete.	
• EXPLOIT-DB:45528 • URL:https://www.exploit-db.com/exploits/45528/ • MISC:https://github.com/pypa/virtualenv/issues/1207	

Vulnerability – Common Vulnerabilities and Exposures (CVE)



The screenshot shows the virtualenv documentation page. The left sidebar contains a navigation menu with links: Installation, User Guide, Reference Guide, Development, and Release History. The main content area is titled "Introduction" and contains the following text:

`virtualenv` is a tool to create isolated Python environments.

The basic problem being addressed is one of dependencies and versions, and indirectly permissions. Imagine you have an application that needs version 1 of Lib Foo, but another application requires version 2. How can you use both these applications? If you install everything into `/usr/lib/python2.7/site-packages` (or whatever your platform's standard location is), it's easy to end up in a situation where you unintentionally upgrade an application that shouldn't be upgraded.

Or more generally, what if you want to install an application *and leave it be*? If an application works, any change in its libraries or the versions of those libraries can break the application.

Also, what if you can't install packages into the global `site-packages` directory? For instance, on a shared host.

In all these cases, `virtualenv` can help you. It creates an environment that has its own installation directories, that doesn't share libraries with other virtualenv environments (and



Exploit – khai thác/cách thức khai thác

- **(động từ):** lợi dụng 1 lỗ hổng để khiến hệ thống phản ứng lại theo 1 cách khác với thiết kế ban đầu.
- **(danh từ):** công cụ, tập các lệnh, hay mã nguồn được dùng để lợi dụng 1 lỗ hổng. Tên gọi khác *Proof of Concept (POC)*.





Fuzzer (danh từ): một công cụ hoặc ứng dụng có chức năng thử tất cả, hoặc 1 loạt các đầu vào không mong muốn cho một hệ thống.

- Mục đích của fuzzer là xác định hệ thống có bug hay không, để tránh bị khai thác mà không cần biết tất cả về hoạt động bên trong của hệ thống.



0day (danh từ): 1 cách thức khai thác 1 lỗ hổng chưa được tiết lộ công khai. Đôi khi cũng được dùng cho khái niệm **lỗ hổng**.

Một lỗ hổng zero-day là 1 lỗi, điểm yếu hoặc bug có trong phần mềm, firmware hoặc phần cứng đã được tiết lộ công khai nhưng chưa được vá. Các nhà nghiên cứu đã tiết lộ về lỗ hổng, và các nhà sản xuất và phát triển đã biết về vấn đề bảo mật đó, nhưng chưa có bản vá hoặc bản cập nhật chính thức để giải quyết.

n-day (danh từ): Một khi lỗ hổng đã được công bố công khai và các nhà sản xuất hay phát triển đã có các bản vá cho nó, lỗ hổng trở thành lỗ hổng đã biết hay n-day.



Khi hacker hoặc **các tác nhân đe dọa phát triển và triển khai các cách thức tấn công** hoặc 1 mã độc thành công để khai thác lỗ hổng khi **nhà sản xuất vẫn đang cố gắng vá lỗ hổng** (hoặc đôi khi, không biết về sự tồn tại của lỗ hổng), được gọi là **zero-day exploit** hoặc tấn công zero-day.

Trong khi các nhà phát triển và nhà cung cấp, cũng như các nhà nghiên cứu và chuyên gia bảo mật, liên tục đầu tư thời gian và nỗ lực để tìm và sửa các lỗi bảo mật, điều tương tự cũng có thể xảy ra đối với các tác nhân đe dọa.



Khai thác các lỗ hổng bảo mật cần nắm chắc được hợp ngữ - assembly, vì hầu hết các cách thức khai thác (exploit) đều cần viết (hoặc chỉnh sửa) mã nguồn dạng hợp ngữ (IA32).

Thanh ghi

- ❑ Cần hiểu cách thanh ghi hoạt động trong bộ xử lý IA32 và cách chúng được thay đổi thông qua các lệnh assembly (đọc, thay đổi...), để khai thác các lỗ hổng.

□ 4 nhóm thanh ghi:

- **General purpose – Mục đích chung:** dùng để thực hiện các phép tính toán thông thường (**EAX, EBX, ECX**). Một thanh ghi mục đích chung khác cần chú ý là **ESP** hay **con trỏ stack**.
- **Segment:** thanh ghi **CS, DS, SS registers**, 16 bits, không giống như các thanh ghi 32 bit khác trong IA32, dùng để theo dõi các segment và cho phép tương thích ngược với các ứng dụng 16-bit.
- **Control – Điều khiển:** điều khiển hàm trong bộ xử lý, 1 trong các thanh ghi quan trọng nhất là **Extended Instruction Pointer (EIP)**, chứa địa chỉ của lệnh mã máy tiếp theo sẽ được thực thi.
- **Khác:** các thanh ghi thêm. Một ví dụ là thanh ghi **Extended Flags (EFLAGS)**, lưu các giá trị tương ứng với kết quả của các phép kiểm tra.

Khi đã hiểu rõ về các thanh ghi, có thể chuyển sang lập trình hợp ngữ :)

Kiến trúc Intel, 32 Bit (IA32 or x86)

INSTRUCTIONS AND DATA

A modern computer makes no real distinction between instructions and data. If a processor can be fed instructions when it should be seeing data, it will happily go about executing the passed instructions. This characteristic makes system exploitation possible. This book teaches you how to insert instructions when the system designer expected data. You will also use the concept of overflowing to overwrite the designer's instructions with your own. The goal is to gain control of execution.

- ❑ Khi thực thi chương trình, các thành phần khác nhau của chương trình sẽ được ánh xạ vào bộ nhớ một cách có tổ chức.
- ❑ Đầu tiên, hệ điều hành tạo 1 vùng địa chỉ để chạy chương trình. Vùng địa chỉ này bao gồm các lệnh thực thi của chương trình cũng như các dữ liệu cần thiết.
- ❑ Tiếp theo, các thông tin được đưa từ file thực thi của chương trình sang vùng địa chỉ vừa tạo. Có 3 dạng segment khác nhau bao gồm: `.text`, `.bss`, and `.data`
 - `.text`: chứa các lệnh thực thi của chương trình, vùng chỉ-đọc.
 - `.data` và `.bss`: cho các biến toàn cục, trong đó `.data` cho các biến static và đã khởi tạo, `.bss` dành cho các biến chưa khởi tạo
- ❑ Stack và heap được khởi tạo:
 - Stack: cấu trúc dữ liệu theo dạng Last In First Out (LIFO)
 - Heap: cấu trúc dữ liệu cho cấp phát động

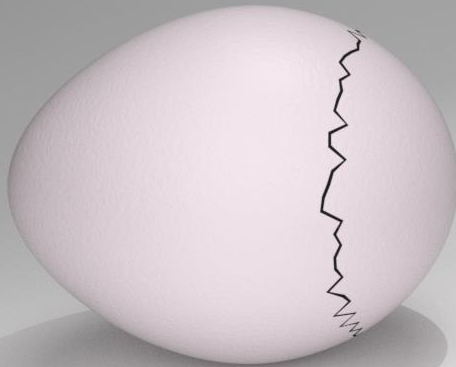
- Stack: phát triển từ địa chỉ cao xuống địa chỉ thấp nếu có dữ liệu được thêm vào stack.
- Heap: phát triển từ địa chỉ thấp lên địa chỉ cao nếu có dữ liệu được thêm vào stack.

↑ Lower addresses (0x08000000)
Shared libraries
.text
.bss
Heap (grows ↓)
Stack (grows ↑)
env pointer
Argc
↓ Higher addresses (0xbfffffff)

Xem thêm về nguyên tắc quản lý bộ nhớ trên Linux: <http://linux-mm.org/>

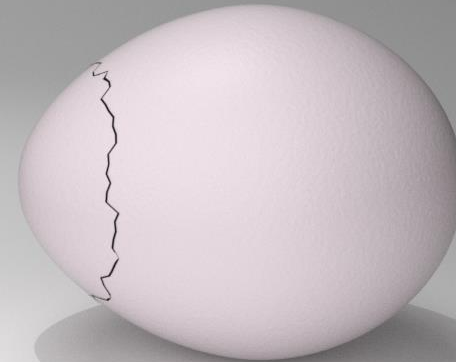
Endianness

The way traditionally
lilliputians broke their boiled eggs
on the larger end



BIG ENDIAN

The way the king then ordered
lilliputians to break their boiled eggs
on the smaller end



Little ENDIAN

getKT.com

Endianness

Value 0x1A2B3C4D

Most Significant Byte First

1A

2B

3C

4D

Big Endian

Least Significant Byte First

4D

3C

2B

1A

Little Endian

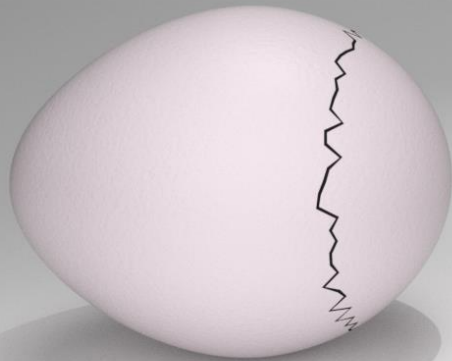
0

1

2

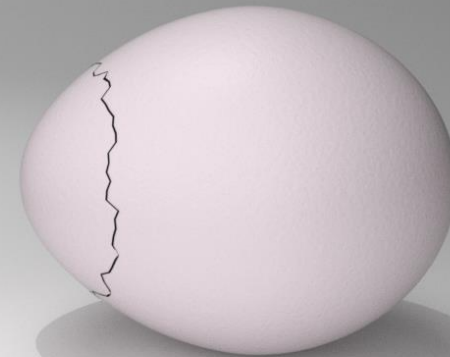
3

Address



BIG ENDIAN

The way traditionally
lilliputians broke their boiled eggs
on the larger end

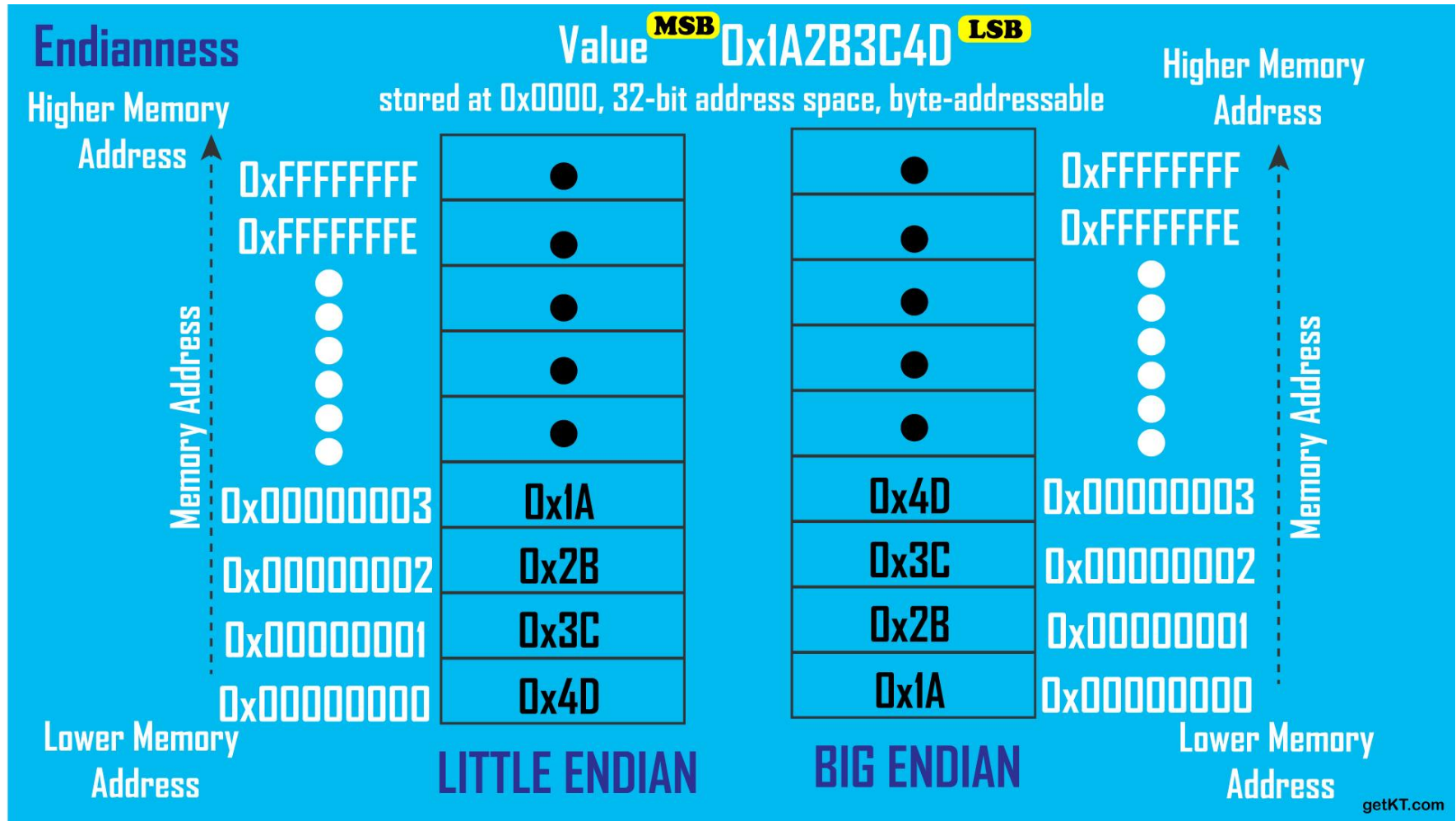


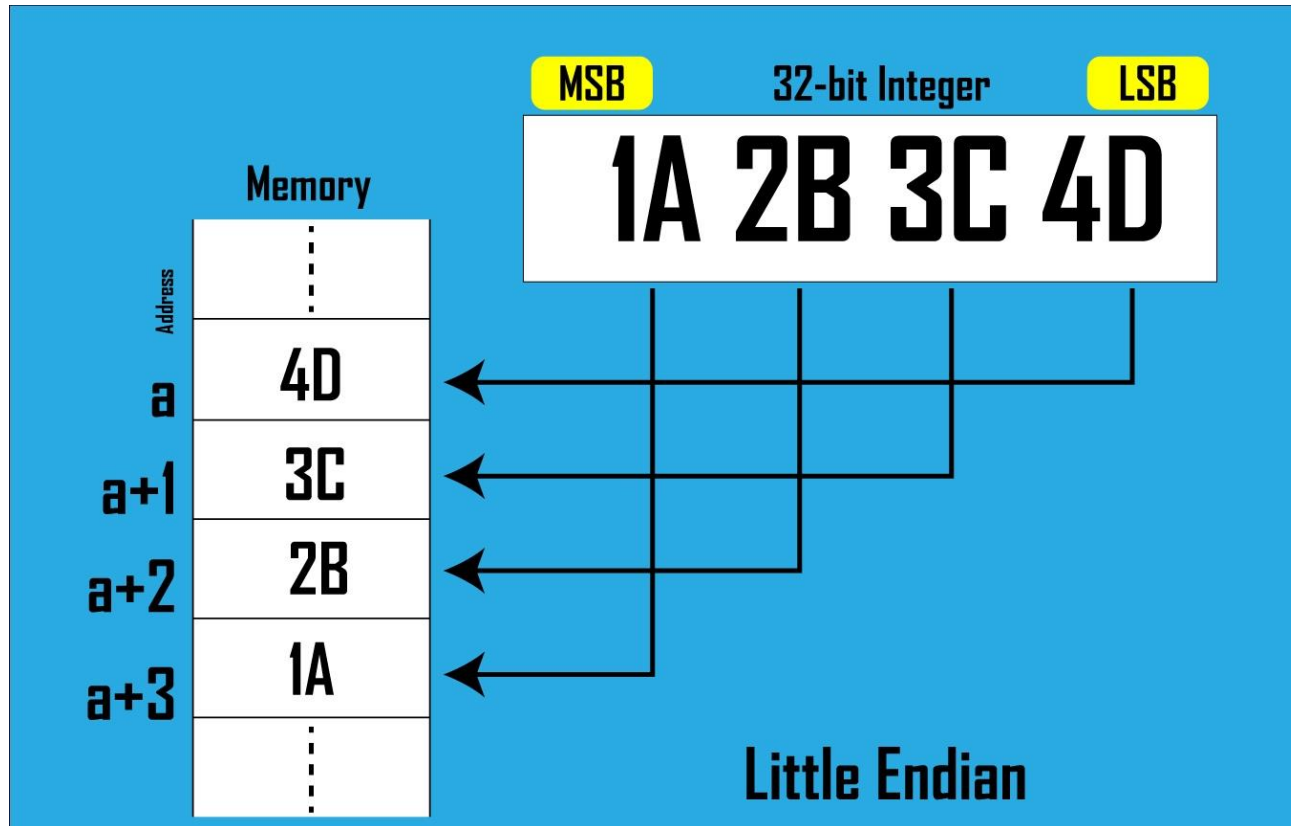
Little ENDIAN

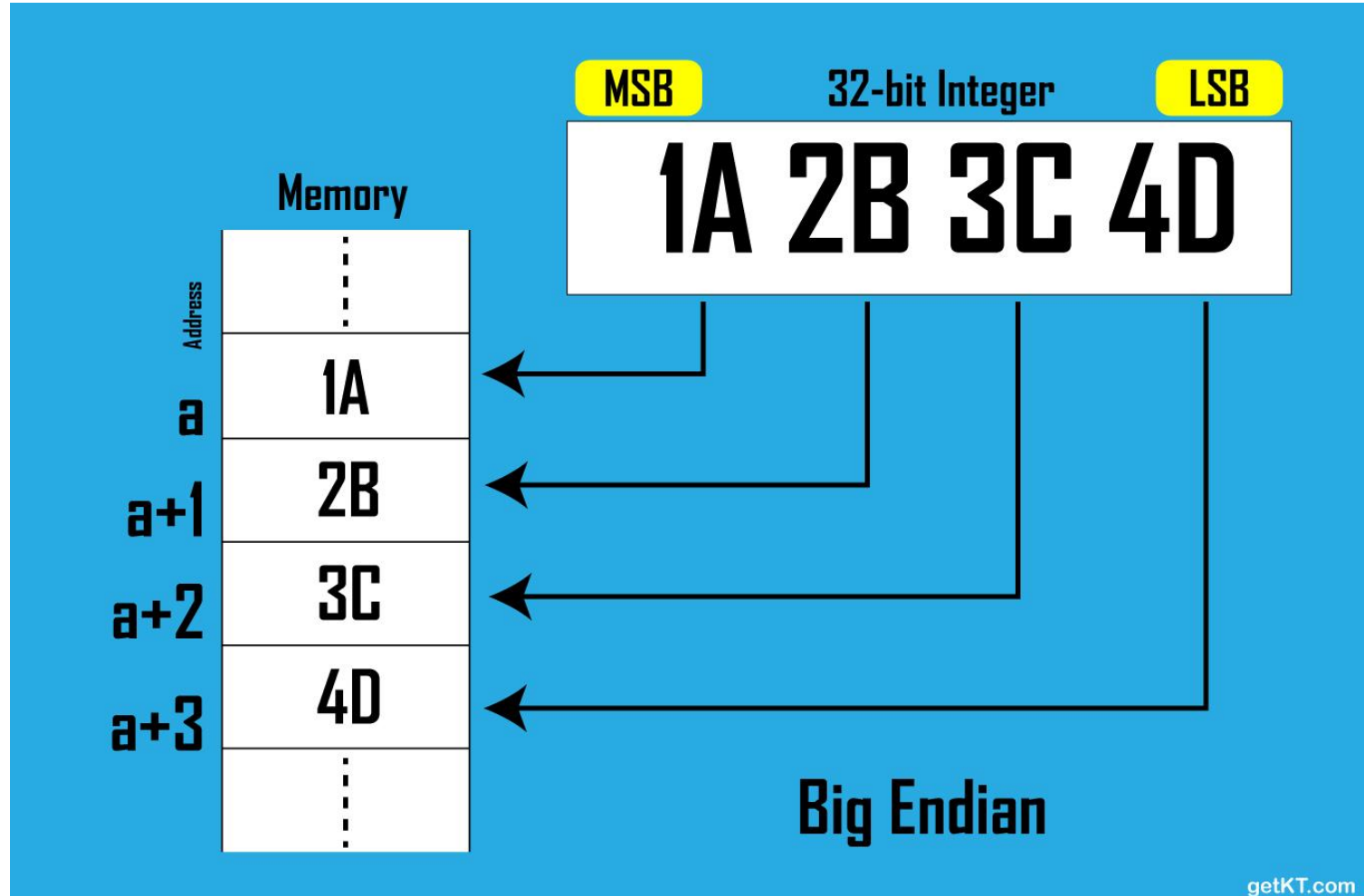
The way the king then ordered
lilliputians to break their boiled eggs
on the smaller end

getKT.com

Big Endian vs. Little Endian









- ❑ C là ngôn ngữ phổ biến nhất cho các ứng dụng server Windows và Linux, thường là các mục tiêu bị tấn công. Do vậy, cần hiểu rõ về ngôn ngữ C.
- ❑ Cần hiểu được cách các mã nguồn C sau khi biên dịch được chuyển sang hợp ngữ như thế nào.
 - Biến
 - Con trỏ
 - Hàm
 - Cấp phát vùng nhớ





Ví dụ: Định nghĩa 1 số nguyên trong C/C++, sau đó dùng chính số nguyên số để đếm.

```
int number;  
... more code ...  
number++;
```

Chuyển sang dạng hợp ngữ:

```
number dw 0  
... more code ...  
mov eax, number  
inc eax  
mov number, eax
```

- Dùng lệnh `dw` để định nghĩa một giá trị số nguyên, `number`.
- Đưa giá trị của `number` vào thanh ghi `EAX`, tăng giá trị trong thanh ghi `EAX` lên 1, sau đó đưa giá trị này về lại số nguyên `number`.





Một lệnh if trong C/C++

```
int number;  
if (number<0)  
{  
    . . .more code . . .  
}
```

Chuyển sang assembly:

```
number dw 0  
mov eax,number  
or eax,eax  
jge label  
<no>  
label :<yes>
```

- Dùng lệnh dw để định nghĩa một giá trị số nguyên, `number`.
- Chuyển giá trị của `number` vào EAX, sau đó nhảy đến nhãn `label` nếu `number` lớn hơn hoặc bằng 0 với lệnh nhảy JGE.





Ví dụ sử dụng mảng trong C:

```
int array[4];  
    . . .more code . . .  
array[2]=9;
```

Chuyển sang dạng assembly:

```
array dw 0,0,0,0  
    . . .more code . . .  
mov ebx,2  
mov array[ebx],9
```

- Định nghĩa 1 mảng
- Dùng thanh ghi `EBX` làm chỉ số index để truy xuất và đưa các giá trị vào mảng.



Ví dụ về hàm trong C

```
int triangle (int width, in height){  
  
    int array[5] = {0,1,2,3,4};  
    int area;  
    area = width * height/2;  
    return (area);  
  
}
```

Hàm trong C sẽ có dạng như thế nào trong assembly?



```

0x8048430 <triangle>:      push    %ebp
0x8048431 <triangle+1>:    mov     %esp, %ebp
0x8048433 <triangle+3>:    push    %edi
0x8048434 <triangle+4>:    push    %esi
0x8048435 <triangle+5>:    sub     $0x30, %esp
0x8048438 <triangle+8>:    lea     0xffffffffd8(%ebp), %edi
0x804843b <triangle+11>:   mov     $0x8049508, %esi
0x8048440 <triangle+16>:   cld
0x8048441 <triangle+17>:   mov     $0x30, %esp
0x8048446 <triangle+22>:   repz   movsl    %ds:( %esi), %es:( %edi)
0x8048448 <triangle+24>:   mov     0x8(%ebp), %eax
0x804844b <triangle+27>:   mov     %eax, %edx
0x804844d <triangle+29>:   imul    0xc(%ebp), %edx
0x8048451 <triangle+33>:   mov     %edx, %eax
0x8048453 <triangle+35>:   sar     $0x1f, %eax
0x8048456 <triangle+38>:   shr     $0x1f, %eax
0x8048459 <triangle+41>:   lea     (%eax, %edx, 1), %eax
0x804845c <triangle+44>:   sar     %eax
0x804845e <triangle+46>:   mov     %eax, 0xffffffffd4(%ebp)
0x8048461 <triangle+49>:   mov     0xffffffffd4(%ebp), %eax
0x8048464 <triangle+52>:   mov     %eax, %eax
0x8048466 <triangle+54>:   add     $0x30, %esp
0x8048469 <triangle+57>:   pop     %esi
0x804846a <triangle+58>:   pop     %edi
0x804846b <triangle+59>:   pop     %ebp
0x804846c <triangle+60>:   ret
    
```

Công cụ sử dụng: **gdb**

▪ Chức năng chính: nhân 2 số với lệnh **imul**.

▪ Phần set-up code để khởi tạo stack frame?

Lưu giá trị của **EBP**, lưu lại 1 số thanh ghi khác, trừ **ESP** mở rộng vùng nhớ trên stack cho các biến cục bộ của hàm.

▪ Phần finish code để thu dọn stack?

Khôi phục các thanh ghi đã lưu
Giá trị trả về lưu thanh ghi **EAX**.
Trở về hàm mẹ với **ret**

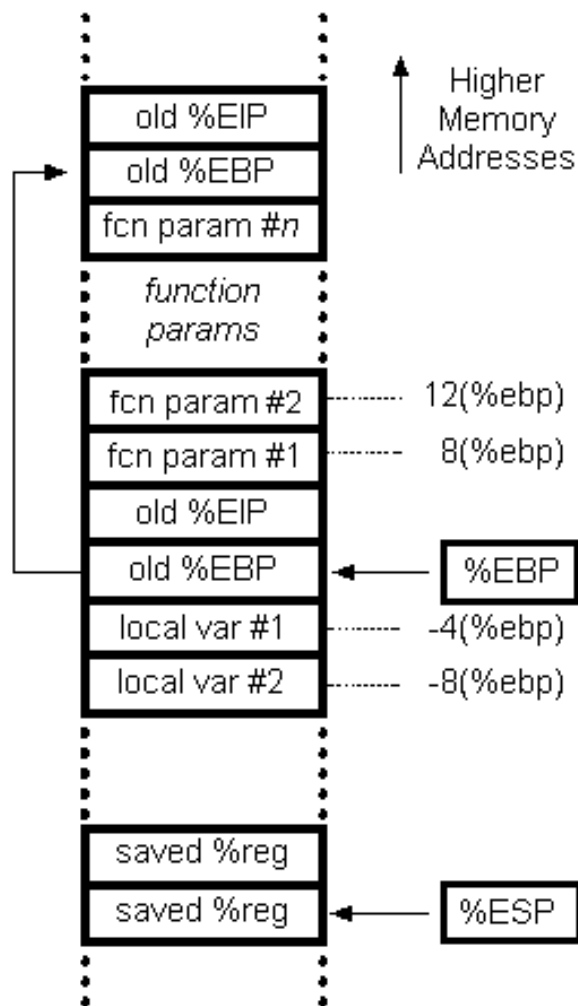


Các thanh ghi dùng trong the stack frame:

- **%ESP - Stack Pointer:** Thanh ghi 32 bit có thể bị thay đổi bởi nhiều lệnh (ví dụ PUSH, POP, CALL, và RET), luôn trỏ đến địa chỉ thấp nhất trong stack.
- **%EBP - Base Pointer:** Thanh ghi 32 bit thường được dùng để tham chiếu đến các tham số và biến cục bộ trong stack frame.
- **%EIP - Instruction Pointer:** Thanh ghi giữ địa chỉ của lệnh tiếp theo sẽ được thực thi, được lưu lại trên stack dưới tác dụng của lệnh CALL. Bất kì câu lệnh “nhảy” nào cũng có thể thay đổi trực tiếp giá trị của EIP.

Các tác vụ được thực hiện khi gọi 1 hàm trong IA32 (__cdecl)

- Đẩy các tham vào stack, theo thứ tự từ phải sang trái trong khai báo C
- Gọi hàm với lệnh **call**
- Lưu và cập nhật giá trị của %ebp
- Lưu các thanh ghi dùng tạm thời
- Cấp phát các biến cục bộ
- Thực hiện chức năng của hàm (*slide kế tiếp*)
- Giải phóng vùng nhớ lưu các biến cục bộ
- Khôi phục các thanh ghi đã lưu
- Khôi phục giá trị của %ebp
- Trở về hàm mẹ
- Thu dọn các tham số đã chuẩn bị



Thực hiện chức năng của hàm:

- Tại thời điểm này, stack frame đã được set up, tham khảo hình bên trái.
- Tất cả **tham số và biến cục bộ** đều được truy xuất dựa trên khoảng cách với **thanh ghi %ebp**.

16(%ebp)	- third function parameter
12(%ebp)	- second function parameter
8(%ebp)	- first function parameter
4(%ebp)	- old %EIP (the function's "return address")
0(%ebp)	- old %EBP (previous function's base pointer)
-4(%ebp)	- first local variable
-8(%ebp)	- second local variable
-12(%ebp)	- third local variable

```
int function_B(int a, int b)
{
    int x, y;

    x = a * a;
    y = b * b;

    return (x+y);
}

int function_A(int p, int q)
{
    int c;
```

```
    c = p * q * function_B(p, p);

    return c;
}

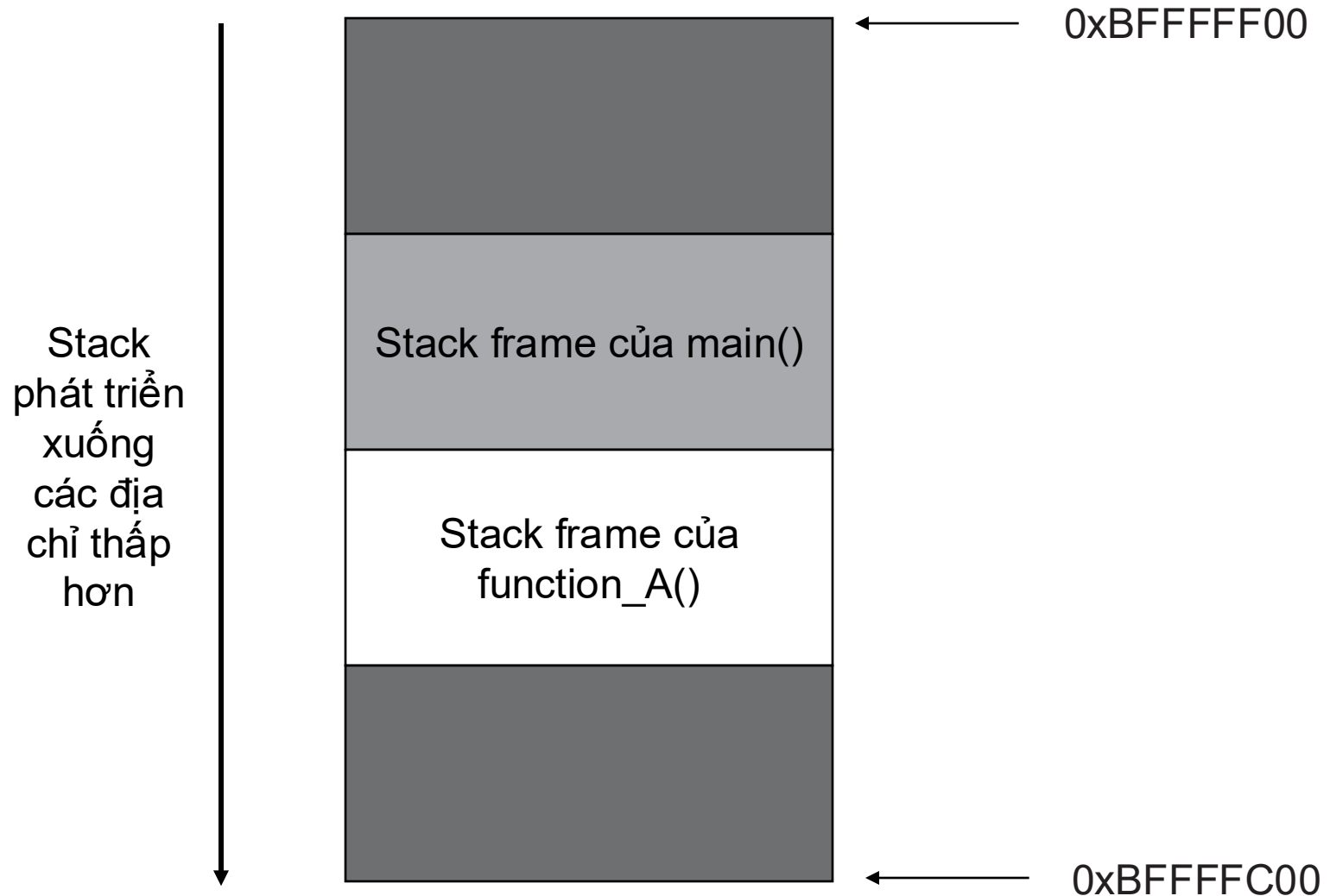
int main(int argc, char **argv, char **envp)
{
    int ret;

    ret = function_A(1, 2);

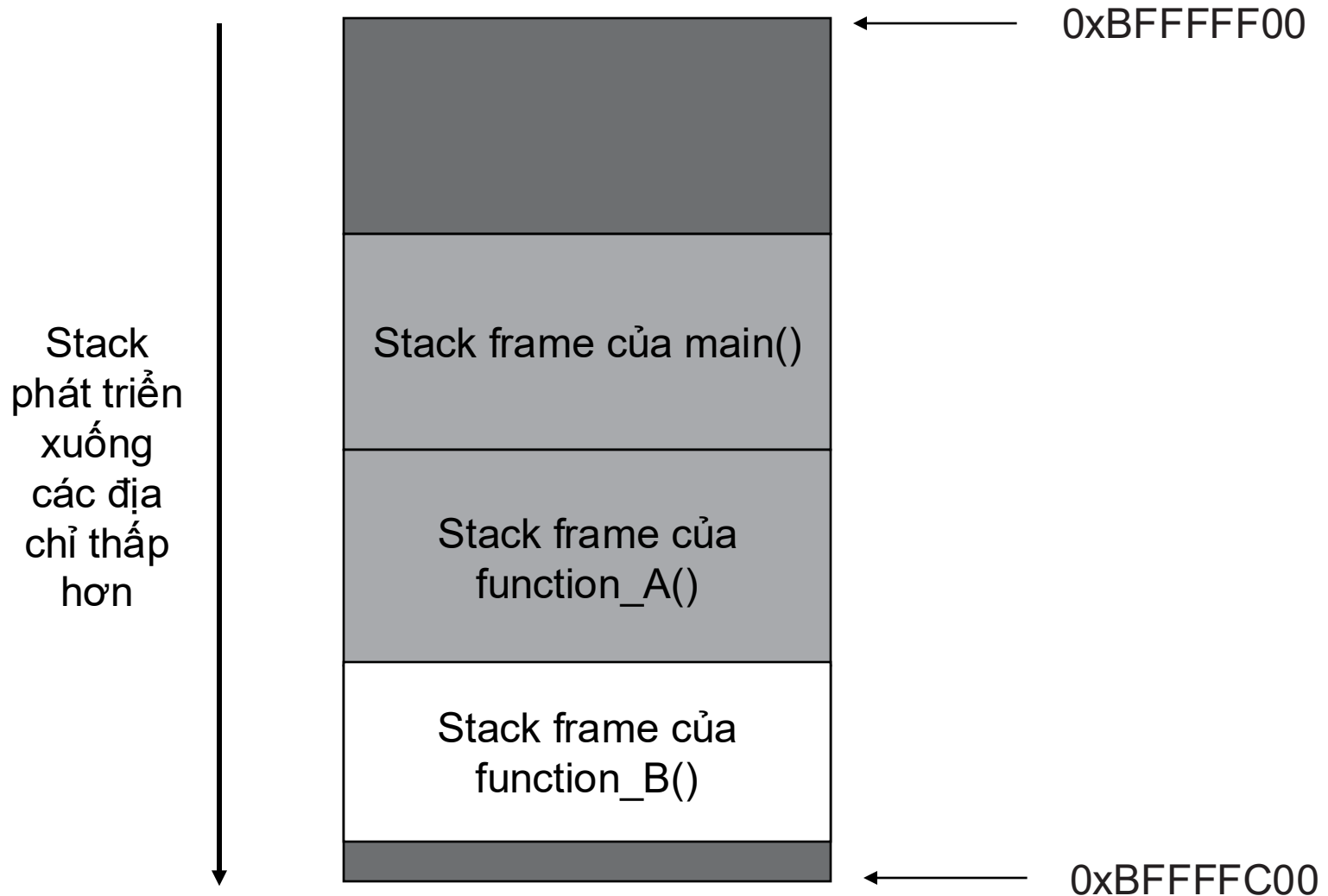
    return ret;
}
```

Luồng thực thi: **main()** → **function_A()** → **function_B()**

Khi **function_A()** được gọi, một stack frame sẽ được cấp phát ở đỉnh stack.



Stack khi thực thi function_A()



Stack khi thực thi function_B()

Stack

- Giới hạn của stack được xác định bằng thanh ghi ESP luôn trỏ đến đỉnh của stack.
- Các lệnh tác động đến stack như PUSH và POP sử dụng ESP để viết vị trí của stack trong bộ nhớ.
- Dữ liệu **được đưa vào stack** với lệnh **PUSH**; dữ liệu được **đưa ra khỏi stack** với lệnh **POP**.

```
push 1
push addr var
```

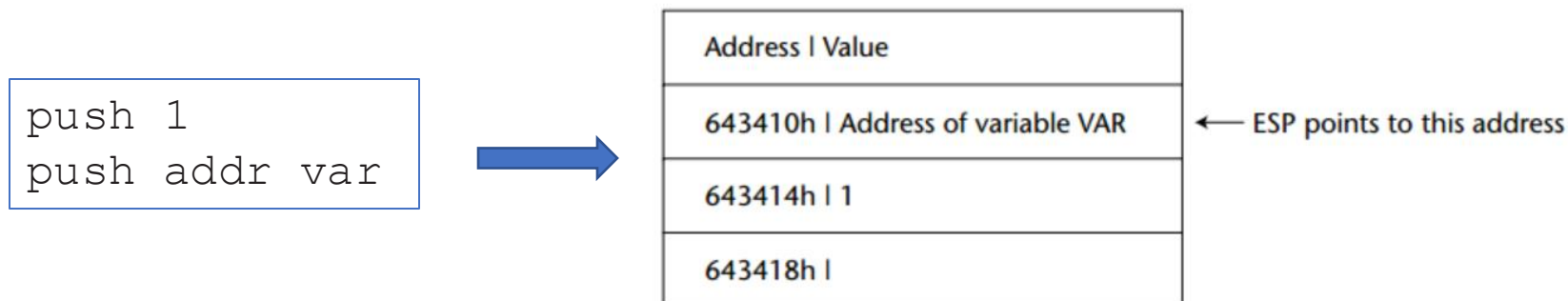


Địa chỉ	Giá trị
0x643418	
0x643414	1
0x643410	Địa chỉ biến var

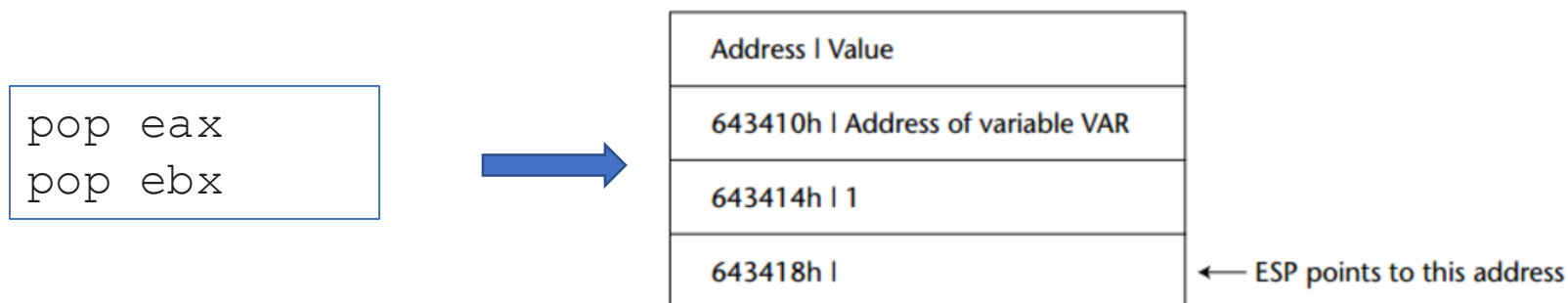
← ESP trỏ đến đây

- 2 lệnh trên lần lượt đưa giá trị 1 và **địa chỉ của biến var** vào stack.
- Thanh ghi ESP sẽ trỏ đến đỉnh stack, là địa chỉ **0x643410**. Các giá trị được đẩy vào stack theo thứ tự thực thi lệnh, do đó giá trị 1 được đẩy vào trước và nằm ở địa chỉ cao hơn so với địa chỉ của biến var được đẩy vào sau.

Khi thực thi lệnh **PUSH instruction**, **ESP giảm xuống 4 đơn vị**, và một giá trị được ghi vào địa chỉ mới đang lưu trong thanh ghi **ESP**.



Lệnh **POP** giúp lấy dữ liệu từ stack, đồng thời **ESP sẽ tăng lên 4 đơn vị**.

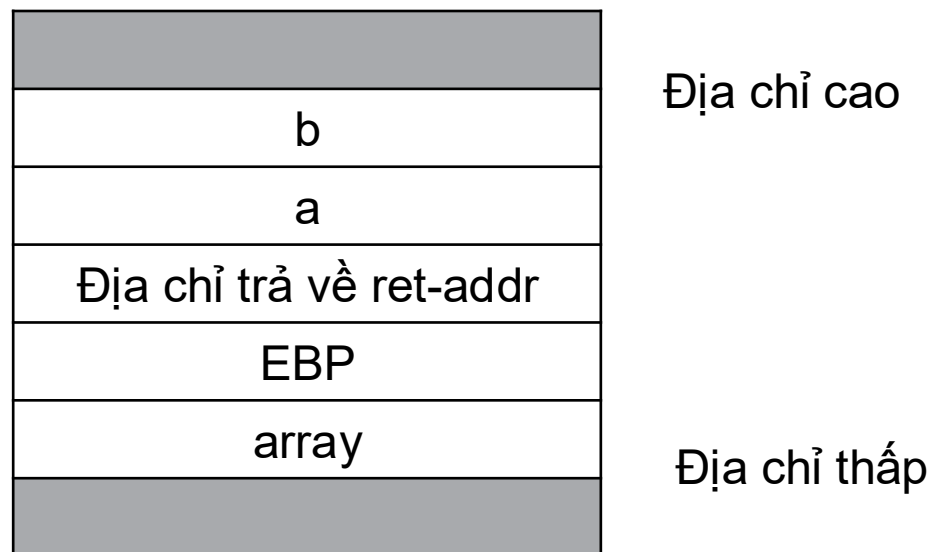


Hàm và Stack

```
void function(int a, int b)
{
    int array[5];
}

main()
{
    function(1,2);

    printf("This is where the return address points");
}
```



Stack khi hàm **function** được gọi

Cách gọi hàm **function**?

```
(gdb) disas main
Dump of assembler code for function main:
0x0804838c <main+0>:    push    %ebp
0x0804838d <main+1>:    mov     %esp,%ebp
0x0804838f <main+3>:    sub     $0x8,%esp
0x08048392 <main+6>:    movl    $0x2,0x4(%esp)
0x0804839a <main+14>:   movl    $0x1, (%esp)
0x080483a1 <main+21>:   call    0x8048384 <function>
0x080483a6 <main+26>:   movl    $0x8048500, (%esp)
0x080483ad <main+33>:   call    0x80482b0 <_init+56>
0x080483b2 <main+38>:   leave
0x080483b3 <main+39>:   ret
End of assembler dump.
```

- Dòng **<main+6>** và **<main+14>**, 2 tham số (0x1) và (0x2) được đưa vào stack.
- Dòng **<main+21>**: gọi hàm function với lệnh **call**.
 - Địa chỉ trả về (ret-addr) hay EIP được đẩy vào stack: 0x80483a6.
 - Chuyển luồng thực thi đến hàm **function**: tại địa chỉ 0x8048384

Xem mã assembly của hàm **function** để hiểu các hoạt động khi mới bắt đầu thực thi hàm

```
(gdb) disas function
Dump of assembler code for function function:
0x08048384 <function+0>:      push    %ebp
0x08048385 <function+1>:      mov     %esp,%ebp
0x08048387 <function+3>:      sub     $0x20,%esp
0x0804838a <function+6>:      leave
0x0804838b <function+7>:      ret
End of assembler dump.
```

- <function+0>: Lưu trữ giá trị EBP hiện tại vào stack.
- <function+1>: Sao chép giá trị hiện tại của ESP vào EBP tại dòng
- <function+3>: Tạo không gian đủ trên stack cho các biến cục bộ, ở đây là biến `array`. Kích thước của `array` là $5 \times 4 = 20$ bytes, nhưng không gian được cấp phát là `0x20` hay 32 bytes.



Intel® 64 and IA-32 Architectures Software Developer's Manual

Volume 2 (2A, 2B, 2C & 2D):
Instruction Set Reference, A-Z

NOTE: The Intel 64 and IA-32 Architectures Software Developer's Manual consists of three volumes: *Basic Architecture*, Order Number 253665; *Instruction Set Reference A-Z*, Order Number 325383; *System Programming Guide*, Order Number 325384. Refer to all three volumes when evaluating your design needs.

REF:

<https://www.intel.de/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-software-developer-instruction-set-reference-manual-325383.pdf>

LEAVE—High Level Procedure Exit

Opcode	Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
C9	LEAVE	NP	Valid	Valid	Set SP to BP, then pop BP.
C9	LEAVE	NP	N.E.	Valid	Set ESP to EBP, then pop EBP.
C9	LEAVE	NP	Valid	N.E.	Set RSP to RBP, then pop RBP.

Instruction Operand Encoding

Op/En	Operand 1	Operand 2	Operand 3	Operand 4
NP	NA	NA	NA	NA

Description

Releases the stack frame set up by an earlier ENTER instruction. The LEAVE instruction copies the frame pointer (in the EBP register) into the stack pointer register (ESP), which releases the stack space allocated to the stack frame. The old frame pointer (the frame pointer for the calling procedure that was saved by the ENTER instruction) is then popped from the stack into the EBP register, restoring the calling procedure's stack frame.

A RET instruction is commonly executed following a LEAVE instruction to return program control to the calling procedure.

See "Procedure Calls for Block-Structured Languages" in Chapter 7 of the **Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 1**, for detailed information on the use of the ENTER and LEAVE instructions.

In 64-bit mode, the instruction's default operation size is 64 bits; 32-bit operation cannot be encoded. See the summary chart at the beginning of this section for encoding data and limits.

REF:

<https://www.intel.de/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-software-developer-instruction-set-reference-manual-325383.pdf>

Một số lệnh trong gdb

- **disassemble main** (disas main) : xem mã assembly của hàm
- **set disassembly-flavor intel**: cấu hình định dạng lệnh intel
- **break main** (b main): đặt breakpoint tại hàm main
- **run**: chạy chương trình
- **stepi** (s), step into: đi đến từng lệnh, có đi vào cụ thể từng hàm
- **nexti** (n), step over: đi đến từng lệnh, tuy nhiên không đi vào cụ thể hàm
- **x/NFU address**: xem giá trị tại địa chỉ nhất định
 - N = number
 - F = format
 - U = unit
 - Ví dụ:
 - x/10xb 0xdeadbeef, xem 10 bytes dạng hex
 - x/xw 0xdeadbeef, xem 1 word dạng hex
 - x/s 0xdeadbeef, xem chuỗi kết thúc bằng null
- **python print 'A'*10**: dùng lệnh python trong gdb